DSA & Advanced DSA Roadmap

Phase 2 — Strings

Learning Objectives

By the end of this phase, learners will be able to:

- Explain how strings are stored and represented internally (ASCII, Unicode, UTF-8).
- Explain why strings are immutable in most languages and the performance implications of that.
- Use StringBuilder-style mutable buffers to avoid repeated string copying.
- Identify and apply core string patterns: character frequency, palindrome checks, anagram detection, two pointers, sliding window, and basic string hashing.
- Solve Easy to Medium level string problems independently.

Implementation & Internals

Core Concepts

- ASCII — fixed 7/8-bit character encoding, the basis for many interview-style assumptions (lowercase a–z, etc.).
- Unicode — a universal character set covering all scripts and symbols.
- UTF-8 — a variable-width encoding of Unicode; why a 'character' is not always one byte.
- String Storage — how strings are laid out in memory (contiguous character arrays).
- String Immutability — why operations like concatenation create new strings rather than modifying in place.
- StringBuilder Internals — how a mutable buffer amortizes the cost of repeated appends.

Implementation Exercises

- Write a function that manually checks whether a character is alphanumeric without using built-in library functions.
- Build a minimal StringBuilder-like class backed by a resizable character array, supporting append and toString.
- Measure and compare the cost of building a long string via repeated += concatenation vs. a builder/buffer approach.
- Implement a function to manually encode a string of ASCII characters into a UTF-8 byte sequence (ASCII range only, for simplicity).

Pattern Mastery

Pattern 1: Character Frequency

Concepts

- Counting character occurrences with a frequency array or hash map.
- Fixed-size frequency arrays (e.g. size 26) vs hash maps for general Unicode.

Practice Problems

- Count occurrences of each character in a string.
- Find the most frequent character.
- Check if a ransom note can be built from a magazine's letters.

LeetCode

- Ransom Note — leetcode.com/problems/ransom-note
- First Unique Character in a String — leetcode.com/problems/first-unique-character-in-a-string

Pattern 2: Palindrome

Concepts

- Checking palindromes via two pointers from both ends.
- Expand-around-center technique for substrings.
- Handling case-insensitivity and non-alphanumeric characters.

Practice Problems

- Check if a string is a palindrome.
- Check if a string can become a palindrome after deleting at most one character.
- Find the longest palindromic substring within a string.

LeetCode

- Valid Palindrome — leetcode.com/problems/valid-palindrome
- Valid Palindrome II — leetcode.com/problems/valid-palindrome-ii
- Longest Palindromic Substring — leetcode.com/problems/longest-palindromic-substring

Pattern 3: Anagram

Concepts

- Two strings are anagrams if their character frequency maps are identical.
- Sorting-based detection vs frequency-count-based detection.
- Grouping multiple strings by their anagram signature.

Practice Problems

- Check if two strings are anagrams of each other.
- Group a list of strings into sets of anagrams.
- Find all starting indices of anagrams of a pattern within a larger string.

LeetCode

- Valid Anagram — leetcode.com/problems/valid-anagram
- Group Anagrams — leetcode.com/problems/group-anagrams
- Find All Anagrams in a String — leetcode.com/problems/find-all-anagrams-in-a-string

Pattern 4: Two Pointers

Concepts

- Opposite-direction pointers for palindrome and reversal checks.
- In-place character swapping without extra storage.

Practice Problems

- Reverse a string in place.
- Check if one string is a subsequence of another using two pointers.
- Find the longest common prefix among an array of strings.

LeetCode

- Reverse String — leetcode.com/problems/reverse-string
- Longest Common Prefix — leetcode.com/problems/longest-common-prefix

Pattern 5: Sliding Window

Concepts

- Fixed and variable-size windows over a string.
- Maintaining a running frequency map as the window slides.
- Shrinking the window when a constraint (e.g. no repeats) is violated.

Practice Problems

- Find the length of the longest substring without repeating characters.
- Find the longest substring after replacing at most k characters.

LeetCode

- Longest Substring Without Repeating Characters — leetcode.com/problems/longest-substring-without-repeating-characters
- Longest Repeating Character Replacement — leetcode.com/problems/longest-repeating-character-replacement

Pattern 6: String Hashing Basics

Concepts

- Mapping a string to a numeric hash for fast comparison.
- Rolling hash: updating a hash incrementally as a window slides, instead of recomputing from scratch.
- Collision risk and why competitive programmers often use double hashing.

Practice Problems

- Detect all 10-character DNA substrings that occur more than once in a longer sequence (a classic rolling-hash problem).
- Implement a simple rolling hash function and use it to detect a repeated substring of fixed length.

LeetCode

- Repeated DNA Sequences — leetcode.com/problems/repeated-dna-sequences

Note: full string-hashing algorithms (Rabin-Karp pattern matching, polynomial rolling hash with modular arithmetic) are covered in depth in Phase 17 (Competitive Programming & Advanced Optimization). This pattern here is intentionally introductory.

Phase Assessment

Implementation Tasks

- Build a minimal StringBuilder-like mutable buffer from scratch.
- Implement manual character-frequency counting without library helper functions.
- Analyze and explain the complexity cost of naive string concatenation in a loop.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/valid-anagram
- leetcode.com/problems/ransom-note
- leetcode.com/problems/first-unique-character-in-a-string
- leetcode.com/problems/valid-palindrome
- leetcode.com/problems/valid-palindrome-ii
- leetcode.com/problems/reverse-string
- leetcode.com/problems/longest-common-prefix

Medium Problems

Anagram

- leetcode.com/problems/group-anagrams
- leetcode.com/problems/find-all-anagrams-in-a-string

Sliding Window

- leetcode.com/problems/longest-substring-without-repeating-characters
- leetcode.com/problems/longest-repeating-character-replacement

Palindrome

- leetcode.com/problems/longest-palindromic-substring

String Hashing

- leetcode.com/problems/repeated-dna-sequences

> **On scope**
>
> *This phase has no Hard problems assigned by design — string-specific Hard problems (e.g. Regular Expression Matching, Edit Distance) depend on Dynamic Programming, which students have not yet covered (Phase 16). Those problems are deferred to a later cross-phase review once DP is complete, rather than assigned here out of sequence.*

Coding Challenges (Assessment Day)

- Longest Substring Without Repeating Characters
- Group Anagrams
- Longest Palindromic Substring
- Valid Palindrome II
- Repeated DNA Sequences

Expected Outcome

Students should be able to:

- Explain string immutability and its performance implications, and use a builder/buffer to avoid it where needed.
- Recognize common string patterns: character frequency, palindrome, anagram, two pointers, sliding window, and basic hashing.
- Solve unseen string problems using pattern-based thinking rather than memorized solutions.
- Complete Easy and Medium string questions independently.